EXPLORING EXCHANGES AND QUEUES IN RABBITMQ

After configuring the integration with RabbitMQ through Spring Cloud Stream, it's time to try running Dispatcher Service.

First, start a RabbitMQ container. Open a Terminal window, navigate to the folder in your `polar-deployment` repository where you keep your docker-compose.yml file (polar-deployment/docker), and run the following command:

```
$ docker-compose up -d polar-rabbitmq
```

Then open another Terminal window, navigate to the Dispatcher Service project's root folder (dispatcher-service), and run the application as follows:

```
$ ./gradlew bootRun
```

The application logs will already give you a hint of what happened, but for a clearer understanding, let's check the RabbitMQ management console (exposed through port 15672).

Open a browser window and navigate to http://localhost:15672. The credentials are the same that we defined in Docker Compose (`user`/`password`). Then go to the Exchanges section. Figure 10.12 shows a list of default exchanges provided by RabbitMQ and the two exchanges generated by our application: `order-accepted` and `order-dispatched`. Spring Cloud Stream maps them to the `packlabel-in-0` and `packlabel-out-0` bindings respectively. The exchanges are *durable* (denoted by the `D` icon in the management console), meaning that they will survive a broker restart.